

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



LẬP TRÌNH NHÂN LINUX

PHẦN 2. ĐỌC GHI FILE TRONG LINUX

Ngành: Công nghệ thông tin

Chuyên ngành: Kỹ thuật phần
mềm nhúng và di động

Mã số: 52.48.02.01

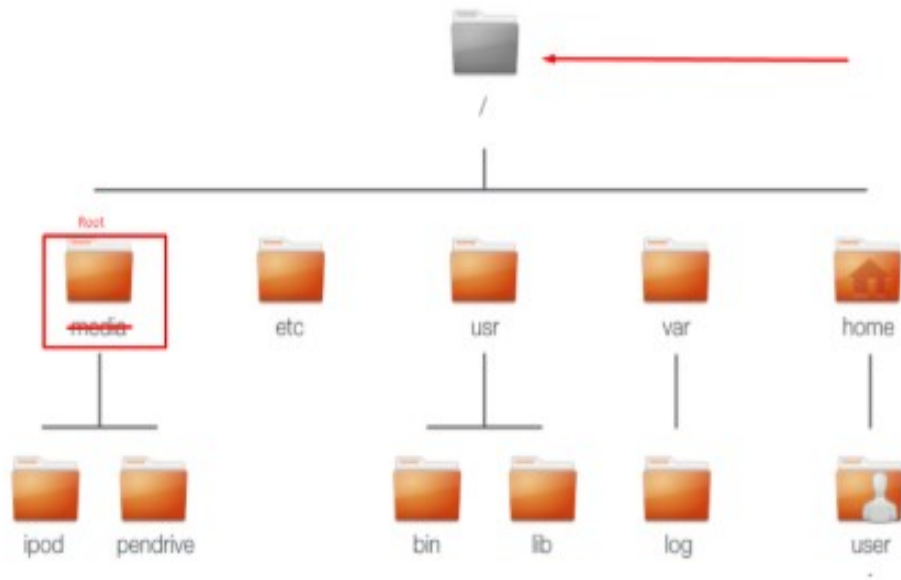
Nội dung

- Phân loại tệp tin
- Bản chất của file
- Các hàm tương tác với file

Định nghĩa

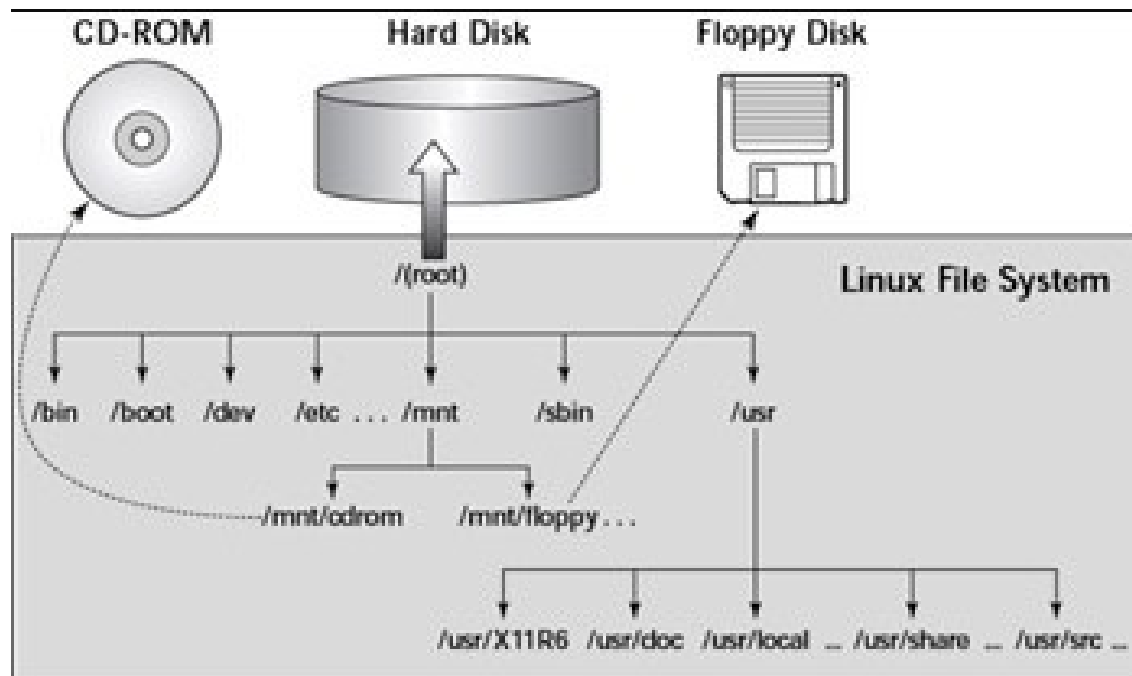
- Là instance của một đối tượng nào đó. Ví dụ process, device, data.
- Linux quản lý mọi thứ thông qua file.
- `struct file include/linux/fs.h`
- Thử đếm số process đang có trạng thái sleep trong hệ thống thông qua /proc folder

Phân loại tệp tin



- Regular file
- Directory
- Các loại file khác

Quá trình khởi tạo file system



5

Read / Write File Operations

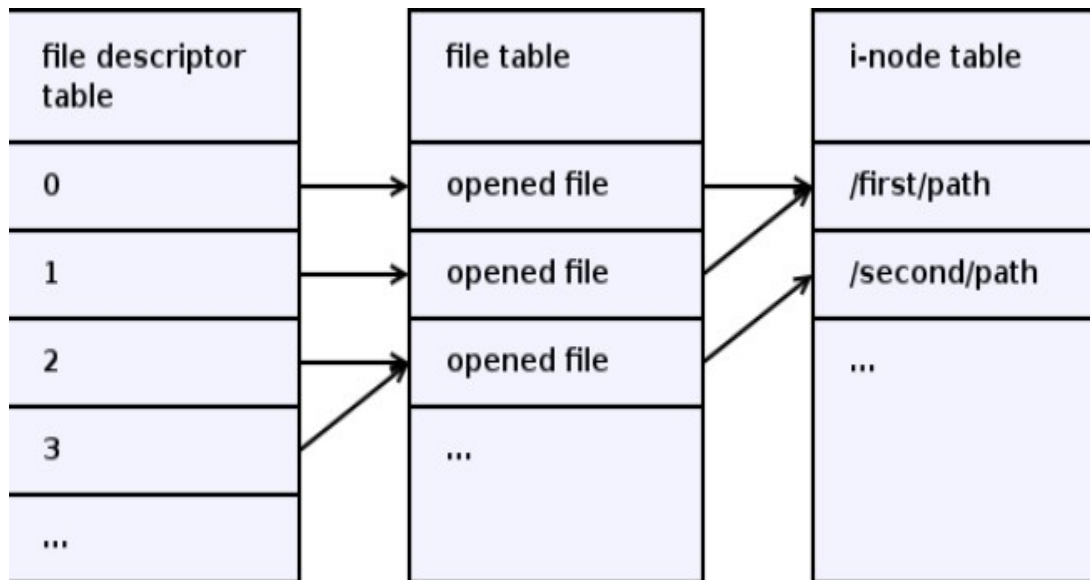


Các hàm cơ bản để tương tác
với file

Các hàm đọc ghi dữ liệu trong file

- Các hàm cơ bản:
 - `int open(const char *pathname, int flags).`
 - `int close(int fd).`
 - `ssize_t read(int fd, void *buf, size_t count).`
 - `ssize_t write(int fd, const void *buf, size_t count)`
 - `off_t lseek(int fd, off_t offset, int whence)`
 - `void sync(void).`

File table của process



- Nếu 2 process cùng ghi vào 1 file có fd bằng nhau thì chuyện gì sẽ xảy ra?

File descriptor

- `include/linux/fs.h`: struct file
- Là struct chứa thông tin về file và các con trỏ trỏ đến các operation của file như read/write.

Block device

- Các thiết bị lưu trữ đa số thuộc loại block device.
- Các thiết bị block device thường quy định kích thước tối thiểu cho mỗi lần đọc.

Cached data của file

- Hệ thống sử dụng 1 phần RAM làm bộ nhớ cached cho việc đọc ghi file.
- Mỗi file khi open sẽ tạo ra 1 inode. Mỗi inode sẽ có trỏ đến vùng nhớ cached riêng của nó.
- Bộ nhớ cached của 1 file có thể được flush theo cách chủ động hoặc bị động

11

Asynchronous File I/O

- Các hàm read/write đều gây block chương trình cho đến khi đọc xong.
- Kỹ thuật khắc phục
 - Sử dụng asynchronous read/write.
 - Tạo 1 thread mới để đọc/ghi

Asynchronous File I/O

```
26 // create the control block structure
27 struct aiocb cb;
28 cb.aio_nbytes = SIZE_TO_READ;
29 cb.aio_fildes = file;
30 cb.aio_offset = 0;
31 cb.aio_buf = buffer;
32 aio_read(&cb);
33 while(aio_error(&cb) == EINPROGRESS);
34 int numBytes = aio_return(&cb);
35 if (numBytes != -1)
36     printf("Success!\n");
```

13

Các hàm đọc ghi thuộc tính của file

- `int stat(const char *restrict pathname, struct stat *restrict buf );`
- `int chmod(const char *pathname, mode_t mode);`
- `int chown(const char *pathname, uid_t owner, gid_t group);`
- `int link(const char *existingpath, const char *newpath);`

14

Các hàm thao tác với thư mục

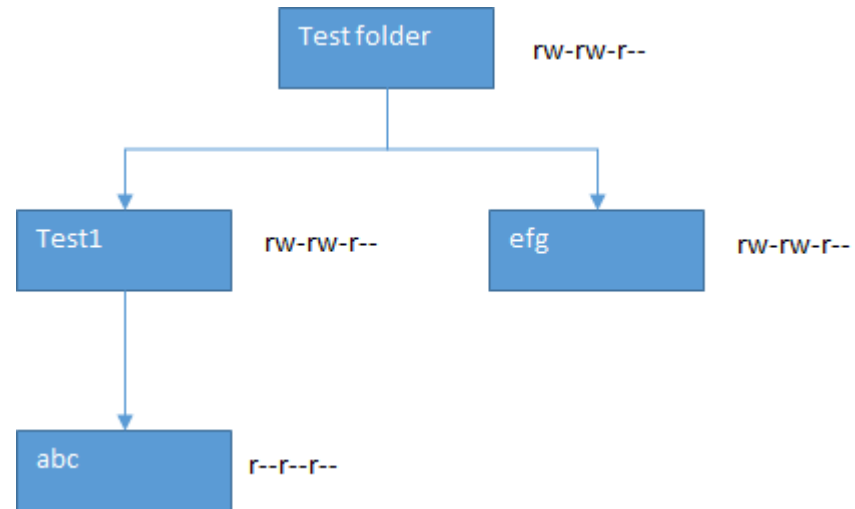
- `int mkdir(const char *pathname, mode_t mode)`
- `DIR *opendir(const char *pathname)`
- `struct dirent *readdir(DIR *dp)`
- `int closedir(DIR *dp)`
- example code

Bài Tập

- Tự tạo 1 file text và đọc nó từ 1 vị trí bất kỳ, tất cả đều sử dụng C.
- Sử dụng hàm `readdir` để in ra thông tin các file trong 1 folder với đường dẫn fixed trong source code. Thông tin gồm có tên, size, modify time.
- Build 1 driver có sẵn hàm `read/write` sau đó viết 1 chương trình C để đọc file và xem kết quả.
 - Source code của driver nằm trong phần code mẫu của bài tập.
 - Step để build: Gõ lệnh `make`, sau đó gõ lệnh `«sudo insmod misc-module»`. Sau khi load thành công sẽ có file `/dev/misc-module` được tạo ra. Chúng ta sẽ đọc ghi vào file đó.

Bài Tập

Dùng C để tạo ra 1 folder có cấu trúc như hình vẽ, trong đó các file có các mode khác nhau. Hiện thị những file có mode là read only.



Bài Tập

- Dùng Makefile build bài tập 5 ra bộ packet cho app bao gồm file binary, file thư viện, file config,...
Viết chương trình setup để install các file vào đúng vị trí mô tả như trong README. Viết chương trình uninstall để gỡ chương trình sau khi đã cài đặt.

HỎI - ĐÁP
